MiraFLIX Project

Autoscaling: Docker Compose vs Kubernetes (k3s)

By Elia Mirafiori & Mattia Sandrini

The MiraFLIX Platform

- **Intelligent Streaming:** A custom-developed streaming platform equipped with an AI-driven recommendation system.
- **Content Processing:** Users upload videos and textual descriptions. Descriptions are asynchronously processed to generate numerical vector embeddings.
- **Semantic Search:** Users can search for content using natural language. The system compares query vectors against stored description vectors.
- **Similarity Matching:** Employs cosine similarity mapping to propose the most highly relevant videos instantly.

Core Technology Stack

Backend & Queues

FastAPI delivers robust async performance for HTTP requests and streaming. **Redis** manages background job queues for heavy embedding generation tasks.

Relational & Vector DB

PostgreSQL + pgvector serves as a solid relational database for video metadata while natively storing and querying the multi-dimensional vector embeddings.

AI Framework

Ollama runs securely as our local execution framework, specifically hosting the **gemmaembedding** model for AI-driven natural language processing.

Project Roles & Infrastructure

Elia Mirafiori

Selected the tech stack and developed the entire platform architecture. Configured the local Docker Compose scenario and drafted the initial performance tests using Grafana k6.

Mattia Sandrini

Configured the Kubernetes (k3s) cluster for high performance. Deployed Grafana k6 and Prometheus for advanced metric collection, and executed all intensive benchmark testing.

Proxmox VE Host

The foundational hypervisor hosted on physical hardware. It enabled the simulation of both a dedicated single host (Docker) and a distributed multi-node environment (k8s).

The Orchestration Contenders

Docker Compose

Local Environment: Deployed on a single VM (VM 204). Designed for localized solutions not anticipating extreme concurrent utilization.

Networking: Handles all services within a single host's boundaries, resulting in zero network overhead (localhost).

Scalability: Hard-limited by the host's physical capacity. In our testing, the worker capacity is fixed at 1.

Kubernetes (k3s)

Distributed Cluster: Deployed across 4 VMs (1 master, 2 workers, and a dedicated NFS serve). Designed for intensive, production-grade workloads.

Networking: Introduces unavoidable network overhead due to the CNI (Flannel) and kube-proxy traffic routing.

Scalability: Unlocks dynamic horizontal scaling, self-healing architectures, and true high availability.

Horizontal Pod Autoscaler (HPA)

The HPA automatically adjusts the number of background worker Pods based on observed custom metrics, ensuring AI tasks never time out under high load.

In our k3s environment, we configured dynamic scaling from **1 to 8 workers** triggered by two strict conditions:

- CPU utilization exceeding **70%**.
- Redis job queue exceeding **5 jobs per worker**.

When demand subsides, HPA gracefully scales down, optimizing cluster resource usage.

Environment Setup & Baseline

Parameter	Docker Compose	Kubernetes (k3s)
Host	VM 204 (Proxmox)	Cluster of 3 VMs + 1 NFS VM
Resources	4 vCPU / 8 GB RAM	6 vCPU / 12 GB RAM Total
Backend Limits	None (Host limits)	CPU: 4, Memory: 1Gi
Worker Scaling	1 Fixed Worker	HPA dynamically 1 → 8
Pre-Test Protocol	Strict baseline reset: queues flushed, workers drained/scaled down.	

The Cost of Distribution (p95 Latency)

Kubernetes pays a network latency tax (1.3x - 5.3x slower) due to CNI routing and distributed node communication.

Docker's localhost bypasses this entirely.

The Search Scenario: Proving HPA

30

Iterations (Docker)

1

Iteration (k3s w/o HPA)

With a single worker, the distributed k8s pipeline (Worker → Ollama → pgvector) exceeds the polling window and times out. Docker completes the iteration only due to zero-latency localhost networking. **This proves that dynamic queue-based HPA scaling is mandatory for distributed pipelines to maintain throughput.**

Stress Test & Resilience

Docker: The Breaking Point

Work Processed: Only **9.7%** of AI embeddings were processed under maximum load due to the fixed single-worker bottleneck.

Catastrophic Failure: In 1 out of 6 runs, the fixed worker was completely overwhelmed, resulting in a dead backend that required a manual container restart.

Kubernetes: Self-Healing

Work Processed: Processed **26.6%** of embeddings (2.7x more work!). The HPA successfully detected the queue spike and scaled workers from 1 to 8.

Resilience: Zero fatal crashes. The distributed system successfully managed the load saturation and self-healed without manual intervention.

Conclusion

Kubernetes trades baseline network latency for massive, mission-critical gains in fault tolerance, automated scalability, and system resilience.

Questions?

Thank you for your attention.

Elia Mirafiori & Mattia Sandrini
Repo: https://github.com/eliamirafiori/cloud_computing_and_distributed_systems/